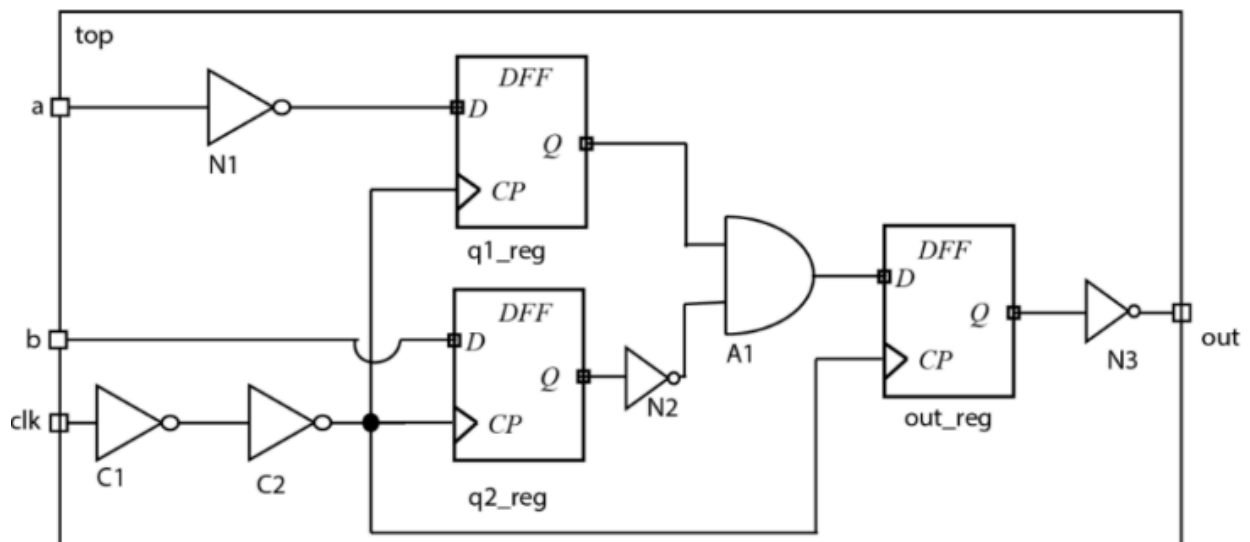


VDF A2 REPORT

Nishchal

2022330

Top Module



Q1 cadence/tempus :

```

[nishchal22330@edatools-server2 nish_vdf]$ csh
[nishchal22330@edatools-server2 nish_vdf]$ source /cadence/cshrc

Welcome to Cadence Tools Suite

[nishchal22330@edatools-server2 nish_vdf]$ tempus -file STA.tcl

Cadence Tempus(TM) Timing Signoff Solution.
Copyright 2020 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:          v20.10-p003_1, built Wed Apr 29 15:56:37 PDT 2020
Options:          -file STA.tcl
Date:             Tue Oct 28 15:30:29 2025
Host:             edatools-server2.iiitd.edu.in (x86_64 w/Linux 2.6.32-754.35.1.el
6.x86_64) (18cores*72cpus*Intel(R) Xeon(R) Gold 6354 CPU @ 3.00GHz 39936KB)
OS:               Red Hat Enterprise Linux Server release 6.10 (Santiago)

License:
    tpsxl   Tempus Timing Signoff Solution XL      20.1   checkout
succeeded

    16 CPU jobs allowed with the current license(s). Use set_multi_c
pu_usage to set your required CPU count.
Change the soft stacksize limit to 0.2%RAM (1033 mbytes). Set global soft_stack_
size limit to change the value.
environment variable TMPDIR is '/tmp/ssv_tmpdir_71305_kslPIt'.
#@ Processing -files option
Sourcing tcl/tk file 'STA.tcl' ...
Scheduling timing library file(s) 'slow.lib' to be loaded when the set_top_modu
le command is issued
Scheduling netlist file(s) '/home/harshit22210/Desktop/nish/syn_net.v' to be loa
ded when the set_top_module command is issued
#% Begin Load MMMC data ... (date=10/28 15:30:38, mem=493.3M)
Number of views requested 1 are allowed with the current licenses... continuing
with set_analysis_view.
#% End Load MMMC data ... (date=10/28 15:30:38, total cpu=0:00:00.0, real=0:00:0
0.0, peak res=493.4M, current mem=493.4M)
Loading view definition file from .ssv_emulate_view_definition_71305.tcl
Reading default_emulate_libset_max timing library '/home/nishchal22330/Desktop/c

```

Q2 top.v (netlist)

```

`timescale 1ns / 1ps

module syn_netlist_top (
    input wire a,
    input wire b,
    input wire clk,|
    output wire out
);

    wire a_n1;           // output of N1
    wire clk_c1;         // after C1
    wire clk_del;        // after C2 (delayed clock)
    wire q1_q;           // q1_reg Q
    wire q2_q;           // q2_reg Q
    wire q2_q_n;         // inverted q2_q (N2)
    wire a1_out;         // output of OR gate A1 (D for out_reg)
    wire out_q;          // out_reg Q
    // ----- Clock inverters (C1, C2) -----
    // clk -> C1 -> clk_c1 -> C2 -> clk_del
    INVX1 u_C1 ( .A(clk), .Y(clk_c1) );
    INVX1 u_C2 ( .A(clk_c1), .Y(clk_del) );

    // ----- Input inverter N1 for 'a' -----
    INVX1 u_N1 ( .A(a), .Y(a_n1) );

    // ----- D flip-flops (positive-edge) -----
    // q1_reg captures a_n1 on rising edge of clk_del
    DFFX1 u_q1_reg ( .D(a_n1), .CK(clk_del), .Q(q1_q) );

    // q2_reg captures b on rising edge of clk_del
    DFFX1 u_q2_reg ( .D(b), .CK(clk_del), .Q(q2_q) );

    // out_reg captures combinational A1 result on rising edge of clk (main clock)
    DFFX1 u_out_reg ( .D(a1_out), .CK(clk), .Q(out_q) );

    // ----- Inverter N2 (inverts q2_q before OR) -----
    INVX1 u_N2 ( .A(q2_q), .Y(q2_q_n) );

    // ----- OR gate A1 (q1_q OR (not q2_q)) -----
    OR2X1 u_A1 ( .A(q1_q), .B(q2_q_n), .Y(a1_out) );

    // ----- Output inverter N3 -----
    INVX1 u_N3 ( .A(out_q), .Y(out) );

endmodule

```

Q3 top.sdc + Timing Report

sdc:

```
create_clock -name clk -period 10 [get_ports clk]

set_input_delay 2 -clock clk [get_ports {a b}]

set_output_delay 2 -clock clk [get_ports out]

set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]
```

Worst Setup :

```
=====
Path 1: MET Late External Delay Assertion
Endpoint:  out      (^) checked with  leading edge of 'clk'
Beginpoint: u_out_reg/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- External Delay                2.000
+ Phase Shift                  10.000
= Required Time                 8.000
- Arrival Time                 0.924
= Slack Time                    7.076
  Clock Rise Edge              0.000
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time      0.000
-----

```

Instance	Arc	Cell	Delay	Arrival Time	Required Time
u_out_reg	CK ^	-	-	0.000	7.076
u_out_reg	CK ^ -> Q v	DFFX1	0.340	0.340	7.416
u_N3	A v -> Y ^	INVX1	0.584	0.924	8.000
-	out ^	-	0.000	0.924	8.000

```
-----
```

Worst hold:


```

Path 1: MET Hold Check with Pin u_out_reg/CK
Endpoint:  u_out_reg/D (^) checked with  leading edge of 'clk'
Beginpoint: u_q1_reg/Q  (^) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
+ Hold                          0.011
+ Phase Shift                   0.000
= Required Time                 0.011
Arrival Time                    0.363
Slack Time                      0.352
  Clock Rise Edge              0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time     0.000
-----
Instance  Arc          Cell  Delay  Arrival  Required
          ^             ^      ^      ^      Time    Time
-----
u_q1_reg  CK ^         -      -      0.000   -0.352
u_q1_reg  CK ^ -> Q ^   DFFX1  0.275  0.275   -0.077
u_A1      A ^ -> Y ^   OR2X1  0.088  0.363    0.011
u_out_reg D ^         DFFX1  0.000  0.363    0.011
-----

```

Q4 clock uncertainty + latency Timing Report

sdc:

```
create_clock -name clk -period 10 [get_ports clk]
set_clock_latency -source -early 0.18 [get_clocks clk]
set_clock_latency -source -late 0.22 [get_clocks clk]

set_clock_latency -early 0.75 [get_clocks clk]
set_clock_latency -late 0.85 [get_clocks clk]

set_clock_uncertainty -setup 0.10 [get_clocks clk]
set_clock_uncertainty -hold 0.05 [get_clocks clk]

set_input_delay 2 -clock clk [get_ports {a b}]
|
set_output_delay 2 -clock clk [get_ports out]

set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]
```

Worst Setup :

```

Path 1: MET Late External Delay Assertion |
Endpoint:  out          (^) checked with  leading edge of 'clk'
Beginpoint: u_out_reg/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
+ Source Insertion Delay        0.180
- External Delay                2.000
+ Phase Shift                  10.000
- Uncertainty                   0.100
= Required Time                 8.080
- Arrival Time                 1.144
= Slack Time                    6.936
  Clock Rise Edge               0.000
+ Clock Network Latency (Ideal) 0.220
= Beginpoint Arrival Time       0.220
-----
Instance  Arc          Cell  Delay  Arrival  Required
          Time         Time
-----
u_out_reg CK ^         -      -      0.220    7.156
u_out_reg CK ^ -> Q v  DFFX1 0.340    0.560    7.496
u_N3      A v -> Y ^  INVX1 0.584    1.144    8.080
-         out ^       -      0.000    1.144    8.080
-----

```

Worst hold:

```

Path 1: MET Hold Check with Pin u_out_reg/CK
Endpoint:  u_out_reg/D (^) checked with  leading edge of 'clk'
Beginpoint: u_q1_reg/Q  (^) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.220
+ Hold                          0.011
+ Phase Shift                   0.000
+ Uncertainty                   0.050
= Required Time                 0.281
Arrival Time                    0.543|
Slack Time                      0.262
  Clock Rise Edge              0.000
+ Clock Network Latency (Ideal) 0.180
= Beginpoint Arrival Time      0.180
-----
Instance  Arc          Cell  Delay  Arrival  Required
          Time         Time
-----
u_q1_reg  CK ^         -      -      0.180    -0.082
u_q1_reg  CK ^ -> Q ^   DFFX1  0.275  0.455    0.193
u_A1      A ^ -> Y ^   OR2X1  0.089  0.543    0.281
u_out_reg D ^         DFFX1  0.000  0.543    0.281
-----

```

Adding realistic clock latency and uncertainty tightened the timing on the same data path. The data delay is unchanged (~0.924 ns), but introducing 0.22 ns launch clock latency pushes the data arrival later (0.924 → 1.144 ns). The setup requirement also shrinks slightly due to 0.10 ns setup uncertainty (Required 8.000 → 8.080 ns). Net effect: worst-path setup slack reduces from **7.076 ns** (original) to **6.936 ns** (new), i.e., **-0.140 ns**. For hold, adding early clock latency and 0.05 ns hold uncertainty tightens the check window, dropping worst-path hold slack from **0.352 ns** to **0.262 ns** (**-0.090 ns**). The path still meets timing, but with less margin after modeling clock effects.

Q5 Double clock Frequency

sdc:


```

create_clock -name clk -period 5 [get_ports clk]

set_input_delay 2 -clock clk [get_ports {a b}]

set_output_delay 2 -clock clk [get_ports out]

set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]

```

Worst Setup :

```

Path 1: MET Late External Delay Assertion
Endpoint:  out          (^) checked with  leading edge of 'clk'
Beginpoint: u_out_reg/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- External Delay                 2.000
+ Phase Shift                   5.000
= Required Time                 3.000
- Arrival Time                  0.924
= Slack Time                    2.076
  Clock Rise Edge                0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time      0.000

```

Instance	Arc	Cell	Delay	Arrival Time	Required Time
u_out_reg	CK ^	-	-	0.000	2.076
u_out_reg	CK ^ -> Q v	DFFX1	0.340	0.340	2.416
u_N3	A v -> Y ^	INVX1	0.584	0.924	3.000
-	out ^	-	0.000	0.924	3.000

Worst hold:

```

.....
Path 1: MET Hold Check with Pin u_out_reg/CK
Endpoint:  u_out_reg/D (^) checked with  leading edge of 'clk'
Beginpoint: u_q1_reg/Q  (^) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
+ Hold                          0.011
+ Phase Shift                   0.000
= Required Time                 0.011
Arrival Time                    0.363
Slack Time                      0.352
  Clock Rise Edge              0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time     0.000
-----
Instance  Arc          Cell  Delay  Arrival  Required
          Time         Time
-----
u_q1_reg  CK ^         -      -      0.000    -0.352
u_q1_reg  CK ^ -> Q ^   DFFX1  0.275  0.275    -0.077
u_A1      A ^ -> Y ^   OR2X1  0.088  0.363     0.011
u_out_reg D ^         DFFX1  0.000  0.363     0.011
-----

```

Cutting the clock period from 10 ns to 5 ns squeezed only the setup window. The data path delay stayed ~0.924 ns, but the required time shrank from 8.000 ns to 3.000 ns (phase-shift equals period, minus the same 2 ns external delay). So setup slack dropped from 7.076 ns → 2.076 ns (a -5.000 ns change), making the same path much closer to critical but still meeting timing. The **hold slack is unchanged at ~0.352 ns** because hold is a same-edge check; with the same early clock, uncertainty, and data path, tightening the period doesn't affect hold.

Q6 Tight Input Delay

sdc:

```
create_clock -name clk -period 10 [get_ports clk]

set_input_delay -max 4 -clock clk [get_ports {a b}]
set_input_delay -min 0.2 -clock clk [get_ports {a b}]

set_output_delay 2 -clock clk [get_ports out]

set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]
```

Setup of all paths :

Path 1: MET Setup Check with Pin u_q1_reg/CK
Endpoint: u_q1_reg/D (^) checked with leading edge of 'clk'
Beginpoint: a (v) triggered by leading edge of 'clk'
Path Groups: {clk}

Other End Arrival Time	0.000
- Setup	0.186
+ Phase Shift	10.000
= Required Time	9.814
- Arrival Time	4.025
= Slack Time	5.789

Clock Rise Edge	0.000
+ Input Delay	4.000
= Beginpoint Arrival Time	4.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
-	a v	-	-	4.000	9.789
u_N1	A v -> Y ^	INVX1	0.025	4.025	9.814
u_q1_reg	D ^	DFFX1	0.000	4.025	9.814

Path 2: MET Setup Check with Pin u_q2_reg/CK

Endpoint: u_q2_reg/D (^) checked with leading edge of 'clk'

Beginpoint: b (^) triggered by leading edge of 'clk'

Path Groups: {clk}

Other End Arrival Time 0.000

- Setup 0.202

+ Phase Shift 10.000

= Required Time 9.798

- Arrival Time 4.000

= Slack Time 5.798

Clock Rise Edge 0.000

+ Input Delay 4.000

= Beginpoint Arrival Time 4.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
-	b ^	-	-	4.000	9.798
u_q2_reg	D ^	DFFX1	0.000	4.000	9.798

Path 3: MET Setup Check with Pin u_q1_reg/CK

Endpoint: u_q1_reg/D (v) checked with leading edge of 'clk'

Beginpoint: a (^) triggered by leading edge of 'clk'

Path Groups: {clk}

Other End Arrival Time 0.000

- Setup 0.096

+ Phase Shift 10.000

= Required Time 9.904

- Arrival Time 4.021

= Slack Time 5.883

Clock Rise Edge 0.000

+ Input Delay 4.000

= Beginpoint Arrival Time 4.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
-	a ^	-	-	4.000	9.883
u_N1	A ^ -> Y v	INVX1	0.021	4.021	9.904
u_q1_reg	D v	DFFX1	0.000	4.021	9.904

Path 4: MET Setup Check with Pin u_q2_reg/CK
 Endpoint: u_q2_reg/D (v) checked with leading edge of 'clk'
 Beginpoint: b (v) triggered by leading edge of 'clk'
 Path Groups: {clk}

Other End Arrival Time	0.000
- Setup	0.109
+ Phase Shift	10.000
= Required Time	9.891
- Arrival Time	4.000
= Slack Time	5.891

Clock Rise Edge	0.000
+ Input Delay	4.000
= Beginpoint Arrival Time	4.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
-	b v	-	-	4.000	9.891
u_q2_reg	D v	DFFX1	0.000	4.000	9.891

Path 5: MET Late External Delay Assertion
 Endpoint: out (^) checked with leading edge of 'clk'
 Beginpoint: u_out_reg/Q (v) triggered by leading edge of 'clk'
 Path Groups: {clk}

Other End Arrival Time	0.000
- External Delay	2.000
+ Phase Shift	10.000
= Required Time	8.000
- Arrival Time	0.924
= Slack Time	7.076

Clock Rise Edge	0.000
+ Clock Network Latency (Ideal)	0.000
= Beginpoint Arrival Time	0.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
u_out_reg	CK ^	-	-	0.000	7.076
u_out_reg	CK ^ -> Q v	DFFX1	0.340	0.340	7.416
u_N3	A v -> Y ^	INVX1	0.584	0.924	8.000
-	out ^	-	0.000	0.924	8.000

Effect of Tighter Input Delay on Setup Slack:

Tightening the **input-delay (setup)** on ports **a, b** from **2 ns** → **4 ns** eats directly into the launch time at the input flops, so every setup path that starts at an input now arrives **2 ns later** and loses exactly **2 ns** of margin. In your reports the input paths show:

- via **a** → **u_q1_reg/D**: slack **5.789 ns** (Required 9.814, Arrival 4.025). With the old 2 ns delay this would have been ≈ 7.789 ns.
- via **b** → **u_q2_reg/D**: slack **5.798 ns** (Req 9.798, Arr 4.000) vs ≈ 7.798 ns before.
- via **a** → **u_q1_reg/D** (alt path): slack **5.883 ns** (Req 9.904, Arr 4.021) vs ≈ 7.883 ns before.

So, making the setup constraint tighter on the inputs reduces setup slack on all input-originating paths by ~ 2.0 ns, while non-input paths (e.g., the output path with 7.076 ns slack) are unaffected.

Q7 Tight output Delay

sdc:

```

create_clock -name clk -period 10 [get_ports clk]

set_input_delay 2 -clock clk [get_ports {a b}]

set_output_delay -max 1.0 -clock clk [get_ports out]

set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]

```

Setup for all paths :

```

Path 1: MET Setup Check with Pin u_q1_reg/CK
Endpoint:  u_q1_reg/D (^) checked with leading edge of 'clk'
Beginpoint: a (v) triggered by leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- Setup                        0.186
+ Phase Shift                  10.000
= Required Time                 9.814
- Arrival Time                 2.025
= Slack Time                    7.789

  Clock Rise Edge              0.000
+ Input Delay                  2.000
= Beginpoint Arrival Time      2.000
-----
  Instance  Arc              Cell  Delay  Arrival  Required
              Time          Time
-----
-           a v              -    -      2.000    9.789
u_N1        A v -> Y ^      INVX1 0.025  2.025    9.814
u_q1_reg    D ^              DFFX1 0.000  2.025    9.814
-----

```



```

-----
Path 2: MET Setup Check with Pin u_q2_reg/CK
Endpoint:  u_q2_reg/D (^) checked with leading edge of 'clk'
Beginpoint: b      (^) triggered by leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- Setup                        0.202
+ Phase Shift                   10.000
= Required Time                 9.798
- Arrival Time                 2.000
= Slack Time                   7.798

Clock Rise Edge                0.000
+ Input Delay                  2.000
= Beginpoint Arrival Time      2.000

```

```

-----
Instance  Arc  Cell  Delay  Arrival  Required
              Time    Time
-----
-          b ^ -    -    2.000    9.798
u_q2_reg  D ^ DFFX1 0.000 2.000    9.798
-----

```

```

Path 3: MET Setup Check with Pin u_q1_reg/CK
Endpoint:  u_q1_reg/D (v) checked with leading edge of 'clk'
Beginpoint: a      (^) triggered by leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- Setup                        0.096
+ Phase Shift                   10.000
= Required Time                 9.904
- Arrival Time                 2.021
= Slack Time                   7.883

Clock Rise Edge                0.000
+ Input Delay                  2.000
= Beginpoint Arrival Time      2.000

```

```

-----
Instance  Arc          Cell  Delay  Arrival  Required
              Time    Time
-----
-          a ^ -    -    2.000    9.883
u_N1      A ^ -> Y v  INVX1 0.021 2.021    9.904
u_q1_reg  D v          DFFX1 0.000 2.021    9.904
-----

```


Path 4: MET Setup Check with Pin u_q2_reg/CK
 Endpoint: u_q2_reg/D (v) checked with leading edge of 'clk'
 Beginpoint: b (v) triggered by leading edge of 'clk'
 Path Groups: {clk}

Other End Arrival Time	0.000
- Setup	0.109
+ Phase Shift	10.000
= Required Time	9.891
- Arrival Time	2.000
= Slack Time	7.891

Clock Rise Edge	0.000
+ Input Delay	2.000
= Beginpoint Arrival Time	2.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
-	b v	-	-	2.000	9.891
u_q2_reg	D v	DFFX1	0.000	2.000	9.891

Path 5: MET Late External Delay Assertion

Path 5: MET Late External Delay Assertion
 Endpoint: out (^) checked with leading edge of 'clk'
 Beginpoint: u_out_reg/Q (v) triggered by leading edge of 'clk'
 Path Groups: {clk}

Other End Arrival Time	0.000
- External Delay	1.000
+ Phase Shift	10.000
= Required Time	9.000
- Arrival Time	0.924
= Slack Time	8.076

Clock Rise Edge	0.000
+ Clock Network Latency (Ideal)	0.000
= Beginpoint Arrival Time	0.000

Instance	Arc	Cell	Delay	Arrival Time	Required Time
u_out_reg	CK ^	-	-	0.000	8.076
u_out_reg	CK ^ -> Q v	DFFX1	0.340	0.340	8.416
u_N3	A v -> Y ^	IN VX1	0.584	0.924	9.000
-	out ^	-	0.000	0.924	9.000

Tightening the **output-delay (setup)** on **out** from **2 ns** → **1 ns** gives your block **more of the clock period** to produce the signal before it leaves the chip. In the timing report, the required time at **out** moves from **8.000 ns** (with 2 ns external delay) to **9.000 ns** (with 1 ns), while the path's data arrival stays the same at **0.924 ns**. Therefore the worst output path's setup slack improves from **7.076 ns** → **8.076 ns (+1.000 ns)**. Only output-terminating paths change; input and internal reg-to-reg paths are unaffected.

Q8

Add multi-cycle path exception for **ONLY** the setup analysis with a path multiplier of 4

on the path showing the worst setup slack.

sdc:

```
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 -clock clk [get_ports {a b}]
set_output_delay 2 -clock clk [get_ports out]

set_multicycle_path 4 -setup -from [get_pins -hierarchical {u_q1_reg/Q} ] -to [get_ports {out}]
set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]
```

Worst Setup :

```

Path 1: MET Late External Delay Assertion
Endpoint:  out          (^) checked with  leading edge of 'clk'
Beginpoint: u_out_reg/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- External Delay                 2.000
+ Phase Shift                   10.000
+ Cycle Adjustment              30.000
= Required Time                 38.000
- Arrival Time                  0.924
= Slack Time                    37.076
  Clock Rise Edge                0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time      0.000
-----
Instance  Arc          Cell  Delay  Arrival  Required
          Time         Time
-----
u_out_reg CK ^         -      -      0.000    37.076
u_out_reg CK ^ -> Q v  DFFX1  0.340  0.340    37.416
u_N3      A v -> Y ^  INVX1  0.584  0.924    38.000
-         out ^         -      0.000  0.924    38.000
-----

```

Worst hold:

```

Path 1: MET Hold Check with Pin u_out_reg/CK
Endpoint:  u_out_reg/D (^) checked with leading edge of 'clk'
Beginpoint: u_q1_reg/Q (^) triggered by leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time      0.000
+ Hold                      0.011
+ Phase Shift               0.000
= Required Time             0.011
Arrival Time                0.363
Slack Time                  0.352
Clock Rise Edge             0.000
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time   0.000
-----
Instance  Arc          Cell  Delay  Arrival  Required
Time      Time
-----
u_q1_reg  CK ^          -    -      0.000   -0.352
u_q1_reg  CK ^ -> Q ^    DFFX1 0.275  0.275   -0.077
u_A1      A ^ -> Y ^    OR2X1 0.088  0.363   0.011
u_out_reg D ^          DFFX1 0.000  0.363   0.011
-----

```

Multi-cycle setup exception (×4) on the worst setup path

The reg-to-output path `u_q1_reg/Q` → `out` is architecturally multi-cycle: the downstream interface only samples this signal every 4 clock cycles

the required time expands from 1 period = 10.000 ns to 4 periods = 40.000 ns; after subtracting the 2.000 ns output delay, the tool reports Required = 38.000 ns with a Cycle Adjustment = 30.000 ns. The data path itself is unchanged (Arrival ≈ 0.924 ns), so setup slack improves from ~7.076 ns to 37.076 ns. Because this exception is setup-only, the hold check is unchanged (same-edge) and still meets timing (hold slack ≈ 0.352 ns), ensuring we don't mask any minimum-delay issues.

Add multi-cycle path exception for the hold analysis also for the above path with a

multiplier of 3.

sdc:


```

create_clock -name clk -period 10 [get_ports clk]

set_input_delay 2 -clock clk [get_ports {a b}]

set_output_delay 2 -clock clk [get_ports out]

set_multicycle_path 4 -setup -from [get_pins u_out_reg/Q ] -to [get_ports out]
set_multicycle_path 3 -hold -from [get_pins u_out_reg/Q ] -to [get_ports out]

|
set_drive 0 [get_ports {a b clk}]

set_load 0.1 [get_ports out]

```

Worst Setup :

```

#####
Path 1: MET Late External Delay Assertion
Endpoint:  out          (^) checked with  leading edge of 'clk'
Beginpoint: u_out_reg/Q (v) triggered by  leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time          0.000
- External Delay                2.000
+ Phase Shift                   10.000
+ Cycle Adjustment              30.000
= Required Time                 38.000
- Arrival Time                  0.924
= Slack Time                    37.076
  Clock Rise Edge               0.000
  + Clock Network Latency (Ideal) 0.000
  = Beginpoint Arrival Time      0.000
-----
Instance  Arc          Cell  Delay  Arrival  Required
          Arc          Cell  Delay  Time     Time
-----
u_out_reg CK ^         -     -     0.000   37.076
u_out_reg CK ^ -> Q v  DFFX1  0.340  0.340   37.416
u_N3      A v -> Y ^   INVX1  0.584  0.924   38.000
-         out ^       -     0.000  0.924   38.000
-----

```

Worst hold:

```
Path 1: MET Hold Check with Pin u_out_reg/CK
Endpoint:  u_out_reg/D (^) checked with leading edge of 'clk'
Beginpoint: u_q1_reg/Q (^) triggered by leading edge of 'clk'
Path Groups: {clk}
Other End Arrival Time      0.000
+ Hold                      0.011
+ Phase Shift               0.000
- Cycle Adjustment          0.000
= Required Time             0.011
Arrival Time                0.363
Slack Time                  0.352
  Clock Rise Edge          0.000
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time   0.000
```

Instance	Arc	Cell	Delay	Arrival Time	Required Time
u_q1_reg	CK ^	-	-	0.000	-0.352
u_q1_reg	CK ^ -> Q ^	DFFX1	0.275	0.275	-0.077
u_A1	A ^ -> Y ^	OR2X1	0.088	0.363	0.011
u_out_reg	D ^	DFFX1	0.000	0.363	0.011

Hold multi-cycle exception (×3) for the same path

Because the setup capture for `u_q1_reg/Q` → `out` is shifted to $k+4$ cycles, we also shift the hold reference to the edge just before the capture, i.e., $k+3$ cycles

This tells STA to check that data launched at cycle k remains stable until the $k+3$ edge (the edge preceding the $k+4$ setup capture), avoiding false/over-tight hold requirements that would exist if the tool still checked hold at $k+1$. Functionally, it preserves realistic minimum-delay checking for the multi-cycle interface while relaxing only what's necessary for correctness. Note that this exception applies only to the `reg`→`port` path; your reported worst hold path is `reg`→`reg` (`u_q1_reg/Q` → `u_out_reg/D`), so its hold slack (~0.352 ns) and “Cycle Adjustment = 0.000” remain unchanged—as expected.